



Escuela Técnica Superior de  
**Ingeniería Informática**

Trabajo Complementos de Bases de Datos

# **DeckBuilder: desarrollo de un gestor de barajas multijuego con C# y MongoDB**

Realizado por

**Borrego González, Adolfo  
Sánchez Carmona, Germán**

**Departamento**  
Lenguajes y Sistemas Informáticos

**Grado**  
Grado en Ingeniería Informática - Ingeniería del Software

**Repositorio del proyecto**  
[github.com/germansanchezz/CBD-GR-24](https://github.com/germansanchezz/CBD-GR-24)

# Índice general

---

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción</b>                                        | <b>1</b>  |
| <b>2</b> | <b>Fase 1: Planificación y Diseño</b>                      | <b>2</b>  |
| 2.1.     | Ideas y Alcance                                            | 2         |
| 2.2.     | Análisis del Flujo de Usuario y Casos de Uso               | 2         |
| 2.3.     | Diseño de la Arquitectura de Datos en MongoDB              | 3         |
| 2.4.     | Stack Tecnológico                                          | 4         |
| 2.5.     | Resultado de la fase                                       | 4         |
| <b>3</b> | <b>Desarrollo e Implementación</b>                         | <b>6</b>  |
| 3.1.     | Implementación del núcleo funcional                        | 6         |
| 3.2.     | Implementación del backend                                 | 6         |
| 3.3.     | Implementación del frontend y flujo operativo              | 9         |
| 3.4.     | Capacidades multijuego y editor visual                     | 11        |
| 3.5.     | Resultados de la fase                                      | 12        |
| <b>4</b> | <b>Documentación y Presentación</b>                        | <b>13</b> |
| 4.1.     | Elaboración de la documentación técnica                    | 13        |
| 4.2.     | Preparación de la presentación                             | 13        |
| 4.3.     | Desarrollo del archivo README: Manual usuario y despliegue | 13        |
| 4.4.     | Resultados finales de la fase                              | 14        |
| <b>5</b> | <b>Conclusiones</b>                                        | <b>15</b> |
| <b>6</b> | <b>Declaración sobre el uso de herramientas de IA</b>      | <b>16</b> |
|          | <b>Bibliografía</b>                                        | <b>18</b> |

# 1. Introducción

---

En los últimos años se ha podido observar un notable crecimiento en la popularidad de los juegos de cartas coleccionables (TCG) como pueden ser Pokémon, Magic: The Gathering y Yu-Gi-Oh! Por esto surge la necesidad de usar una herramienta centralizada que permite la construcción de mazos y gestión de inventario. Este trabajo describe el diseño e implementación de DeckBuilder, una aplicación que busca facilitar e integrar las tareas de los coleccionistas.

El proyecto consiste en el desarrollo de una API REST que permita operaciones CRUD (Create, Read, Update, Delete) sobre barajas e inventarios de cartas, implementado en C# mediante .NET 8 y usando MongoDB como base de datos. La elección de estas tecnologías, más en concreto de MongoDB, permite gestionar datos heterogéneos con estructuras y atributos variables, lo que encaja perfectamente con nuestra propuesta multijuego.

El objetivo principal del proyecto es construir una aplicación capaz registrar cartas obtenidas a través de APIs externas dedicadas, de forma que se garantice lo máximo posible la precisión de la información. De este modo, se busca resolver la necesidad de usar múltiples aplicaciones para gestionar las colecciones de distintas franquicias.

El proyecto se estructura en tres fases:

1. Planificación y Diseño
2. Desarrollo e implementación
3. Documentación y Presentación

Como resultado, se ha obtenido una aplicación funcional que destaca por su capacidad de adaptarse a distintos esquemas de datos gracias al uso de bases de datos documentales.

## 2. Fase 1: Planificación y Diseño

---

### 2.1. Ideas y Alcance

En esta primera fase dejamos establecidas las bases del proyecto. Lo primero que hicimos, fue tener una reunión inicial donde definimos el alcance, mediante este, determinamos que para que el sistema fuera útil, debía ser capaz de manejar varios juegos de cartas distintos sin tener que cambiar la base de datos para cada una de las franquicias.

Durante esta primera reunión, contemplamos diversas formas en las que podíamos orientar el proyecto con las tecnologías que habíamos escogido. Tras verificar que podíamos usar APIs externas para la extracción de las cartas, llegamos a la idea de que poder crear barajas personalizadas era lo que más sentido tenía para implementar el CRUD completo que se nos requería.

### 2.2. Análisis del Flujo de Usuario y Casos de Uso

Para que la aplicación fuera operativa, tuvimos que pensar en un flujo de usuario completo y lógico que proporciona valor y fuera útil para los usuarios, este flujo se documentó con los siguientes pasos:

- **Identificación y Autenticación:** El usuario accede al sistema, lo que permite vincular sus barajas personalizadas a su perfil único almacenado en MongoDB.
- **Consulta y Descubrimiento:** El usuario utiliza el buscador. Este componente realiza peticiones a la API externa de `tcgdex.dev`, devolviendo el listado de cartas candidatas.
- **Persistencia en Colección:** Una vez seleccionada una carta, el sistema CRUD realiza una operación de creación (CREATE) para guardar la referencia y los metadatos específicos.
- **Construcción de Barajas:** El usuario crea un contenedor "Baraja" donde añade y retira cartas. Aquí se aplica la lógica de edición (UPDATE) para modificar cantidades o estados.
- **Ver y Eliminar:** El usuario puede ver sus barajas lo que sería la funcionalidad de (READ) y podría si quisiera eliminarla con la operación de (DELETE).

## 2.3. Diseño de la Arquitectura de Datos en MongoDB

Debido a nuestro interés como equipo en las bases de datos no relacionales, seleccionamos la tecnología de MongoDB, lo que implicó que la arquitectura fuera un punto clave de la planificación. A diferencia de los modelos SQL, optamos por un esquema basado en documentos, que nos permite una total flexibilidad con los atributos.

```
_id: ObjectId('69eddaca2223af44157b5217')
Email: "german@test.com"
PasswordHash: "8D969EEF6ECAD3C29A3A629280E686CF0C3F5D5A86AFF3CA12020C923ADC6C92"
DisplayName: "german"
CreatedAtUtc: 2026-04-26T09:28:42.877+00:00
UpdatedAtUtc: 2026-04-26T09:28:42.877+00:00
```

Figura 2.1: Modelo datos User (MongoDB)

```
_id: ObjectId('69ee2d71627c88bc04b782f5')
Name: "Magic Vampiros"
Description: ""
GameType: "magic"
OwnerUserId: ObjectId('69eddaca2223af44157b5217')
Cards: Array (6)
  0: Object
    CardId: "fee330e7-bb67-4379-856f-f99282da6a9f"
    Name: "Vampiro de la Baronía"
    ImageUrl: "https://cards.scryfall.io/normal/front/f/e/fee330e7-bb67-4379-856f-f99..."
    Properties: Array (2)
      0: "Criatura"
      1: "Negro"
    Quantity: 2
  1: Object
    CardId: "810d072f-9140-46ce-9ea5-5c8326a0535a"
    Name: "Vampiro de Refugio Cornisa"
    ImageUrl: "https://cards.scryfall.io/normal/front/8/1/810d072f-9140-46ce-9ea5-5c8..."
    Properties: Array (3)
      0: "Criatura"
      1: "Negro"
      2: "Blanco"
    Quantity: 2
  2: Object
  3: Object
  4: Object
  5: Object
CreatedAtUtc: 2026-04-26T15:21:21.192+00:00
UpdatedAtUtc: 2026-04-26T15:22:08.999+00:00
```

Figura 2.2: Modelo datos Deck (MongoDB)

```

_id: ObjectId('69ee2d83627c88bc04b782f9')
UserId: ObjectId('69eddaca2223af44157b5217')
GameType: "magic"
ExternalCardId: "41189401-8741-4388-aae6-5514600101d1"
Name: "Vampiro de Guul Draz"
ImageUrl: "https://cards.scryfall.io/normal/front/4/1/41189401-8741-4388-aae6-551..."
SetName: "Zendikar"
Rarity: "common"
TypeLine: "Creature — Vampire Rogue"
SearchTags: Array (1)
MainText: "As long as an opponent has 10 or less life, this creature gets +2/+1 a..."
QuantityOwned: 1
QuantityInDecks: 1
AddedAtUtc: 2026-04-26T15:21:39.567+00:00
UpdatedAtUtc: 2026-04-26T15:21:39.769+00:00

```

**Figura 2.3:** Modelo datos User\_Card (MongoDB)

## 2.4. Stack Tecnológico

El requisito principal del proyecto era construir una API REST que permitiese trabajar con una base de datos NoSQL en MongoDB. Para este proyecto en concreto, teníamos la restricción de que la tecnología a utilizar debía ser C#, por lo que construimos el backend en .NET 8. Este se encarga de la lógica de negocio y exponer la API.

Por otro lado, se decidió incluir un frontend mediante React + Vite + TypeScript. Aunque no era requisito obligatorio, pensamos que facilitaría en gran medida la interacción de los usuarios con la API.

En cuanto a infraestructura, el proyecto se desplegó en servicios cloud:

- Render para backend y frontend.
- MongoDB Atlas para alojar la base de datos en entorno gestionado.

Por tanto la aplicación mantiene el núcleo requerido en MongoDB y C#, añadiendo una arquitectura web completa que queda preparada para su uso en un entorno real de producción.

## 2.5. Resultado de la fase

En esta fase hemos podido establecer una buena base para comenzar el desarrollo de las funcionalidades del proyecto. Durante esta etapa han quedado definidos de forma clara los objetivos del sistema, el alcance y el flujo de usuario, asegurándonos de esta forma que la aplicación cubre todas las necesidades esenciales.

Como resultado de esta fase, hemos obtenido una visión clara del funcionamiento que queremos para el sistema, incluyendo la interacción entre el usuario, con la aplicación y con las fuentes externas de datos. Además, se ha

quedado definido el stack tecnológico. Esto nos ha facilitado la transición a la fase de desarrollo, donde hemos podido implementar el sistema de una manera más eficiente y con menor riesgo de errores en la estructura.

## 3. Desarrollo e Implementación

---

### 3.1. Implementación del núcleo funcional

Esta segunda etapa del proyecto se centró en la transformación de los diseños y esquemas previos en un producto de software funcional. El objetivo primordial consistió en construir una base tecnológica sólida que cubriera el ciclo de vida completo del usuario dentro de la aplicación: desde el proceso de autenticación inicial hasta la gestión avanzada de sus barajas.

Para asegurar la calidad del software, la implementación se llevó a cabo de forma incremental y modular. Este enfoque, basado en el cumplimiento de hitos concretos y el uso de commits atómicos, permitió validar de manera constante cada funcionalidad antes de proceder a la siguiente, minimizando el riesgo de errores estructurales.

### 3.2. Implementación del backend

En el entorno de backend, se estructuró una API bajo el entorno de .NET 8 diseñada para soportar operaciones sobre las barajas de usuarios autenticados. Durante este proceso, se priorizó la separación de responsabilidades mediante el uso de DTOs y validaciones independientes, lo que mejora la mantenibilidad del código y evita el acoplamiento entre la lógica de negocio y la capa de transporte de datos.

Sobre esta arquitectura se consolidó el sistema CRUD completo para la entidad de barajas: creación (CREATE), lectura (READ), actualización (UPDATE) y borrado (DELETE). La lógica de actualización fue algo más compleja, ya que incluye la persistencia dinámica de cartas dentro de cada mazo y la implementación de reglas de dominio específicas, como el control de copias máximas permitidas y el tamaño total de la baraja (60 cartas en total, con límites por nombre según juego).

Para garantizar la estabilidad de los datos externos se optimizó la comunicación con los proveedores de cartas mediante el desarrollo de un mejor cliente HTTP. Se realizaron ajustes pequeños en los algoritmos de búsqueda para asegurar que las respuestas obtenidas desde fuentes como Pokémon o Magic fueran consistentes y estuvieran listas para su procesamiento en el servidor.

Los extractos de código [3.1](#), [3.2](#) y [3.3](#) muestran, respectivamente, un ejemplo de modelo, un endpoint de controlador y una validación de reglas de baraja.

En el primer fragmento se observa cómo se modela una baraja en MongoDB con sus campos clave: nombre, tipo de juego, lista de cartas y marca temporal de actualización.



---

```
1 public sealed class Deck
2 {
3     public string Name { get; set; } = string.Empty;
4     public string GameType { get; set; } = DeckGameTypes.Pokemon;
5     public List<DeckCard> Cards { get; set; } = new();
6     public DateTime UpdatedAtUtc { get; set; } = DateTime.UtcNow;
7 }
```

---

### Extracto de código 3.1: Fragmento de modelo Deck (C#)

El segundo fragmento resume el flujo básico del endpoint de creación: validar la entrada, persistir en la colección y devolver la respuesta HTTP con el recurso creado.

---

```
1 [HttpPost]
2 public async Task<IActionResult> CreateDeck([FromBody] CreateDeckRequest
   request)
3 {
4     if (!DeckRequestValidator.TryValidateCreate(request, out var error))
5         return BadRequest(new { message = error });
6
7     await GetDecksCollection().InsertOneAsync(deck);
8     return Created($"/api/decks/{deck.Id}", deck);
9 }
```

---

### Extracto de código 3.2: Fragmento de controlador para crear baraja (C#)

El tercer fragmento recoge dos de las reglas más relevantes del dominio: límite total de cartas por baraja y número máximo de copias por nombre.

---

```
1 if (totalCards > MaxCardsPerDeck)
2 {
3     errorMessage = $"Una baraja no puede superar {MaxCardsPerDeck} cartas
   .";
4     return false;
5 }
6
7 if (copiesByName[normalizedName] > maxCopiesPerName)
8 {
9     errorMessage = $"No puedes tener mas de {maxCopiesPerName} copias con
   el mismo nombre.";
10    return false;
11 }
```

---

### Extracto de código 3.3: Fragmento de validación de reglas de baraja (C#)

La API desarrollada sigue un diseño REST, estructurado en torno a recursos como usuarios, barajas y cartas. Esta organización permite una separación clara de responsabilidades y facilita la escalabilidad del sistema.

## Documentación de la API

Los endpoints implementados cubren tanto la autenticación como la gestión completa de barajas, la colección de cartas del usuario y la comprobación del servicio.

### Comprobación del servicio

- GET /: estado básico del backend.
- GET /api/health/mongo: comprobación de conectividad con MongoDB.

### Autenticación

- POST /api/auth/register: registro nuevos usuarios.
- POST /api/auth/login: autenticación de usuarios existentes.
- DELETE /api/auth/me: eliminación de la cuenta autenticada y sus datos asociados.

### Gestión de barajas (Decks)

- GET /api/decks: obtención de todas las barajas del usuario autenticado.
- GET /api/decks/{deckId}: consulta detallada de baraja específica.
- POST /api/decks: creación de nueva baraja.
- PUT /api/decks/{deckId}: actualización de una baraja, incluyendo modificar cartas.
- DELETE /api/decks/{deckId}: eliminación de una baraja.

### Búsqueda de cartas

- GET /api/cards/search: búsqueda de cartas en fuentes externas.

### Gestión de colección de cartas (User Cards)

- GET /api/user-cards: obtención de la colección de cartas del usuario, con filtros opcionales.
- GET /api/user-cards/stats: obtención de estadísticas de colección, con posibilidad de filtrar por juego.
- POST /api/user-cards: inserción de nuevas cartas en colección.

- POST /api/user-cards/{userCardId}/quantity: ajuste incremental de copias carta.
- DELETE /api/user-cards/{userCardId}: eliminación de carta de colección.

Este conjunto de endpoints constituye la base del sistema CRUD ampliado, no solo sobre barajas sino también sobre la colección personal del usuario. La inclusión de endpoints de estadística y búsqueda externa aporta un valor añadido al sistema.

Su funcionamiento detallado y configuración se encuentran descritos en el archivo README del repositorio, donde se abordan aspectos como la lógica de interacción, validaciones y comunicación con la base de datos.

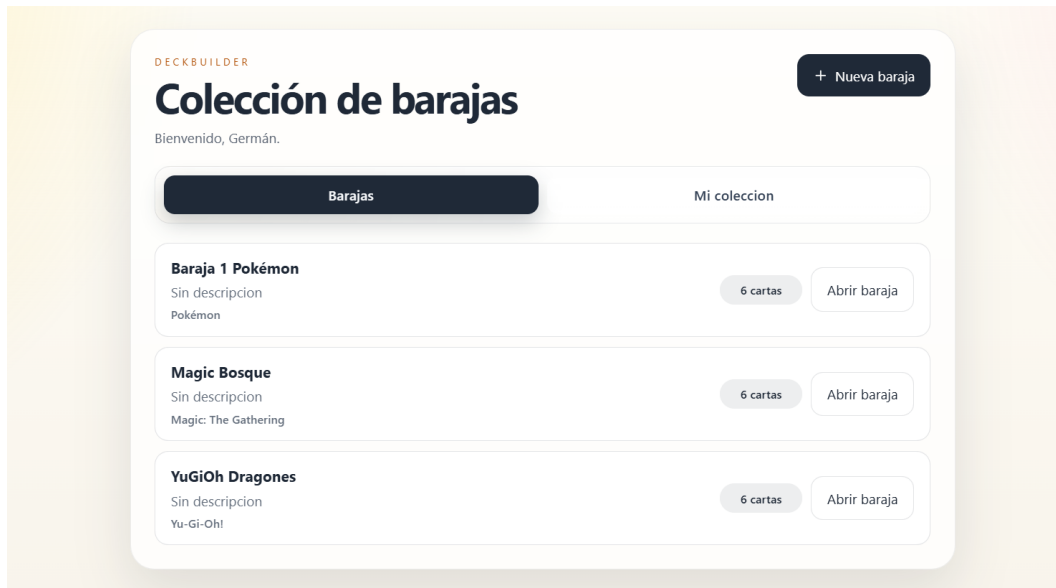
### 3.3. Implementación del frontend y flujo operativo

Paralelamente, el desarrollo de la interfaz de usuario evolucionó desde prototipos estáticos hacia un flujo operativo completo y dinámico. Se implementó un sistema de gestión de sesiones que permite el cambio fluido entre las vistas de registro e inicio de sesión, garantizando que el usuario acceda únicamente a su colección privada almacenada en MongoDB una vez identificado.

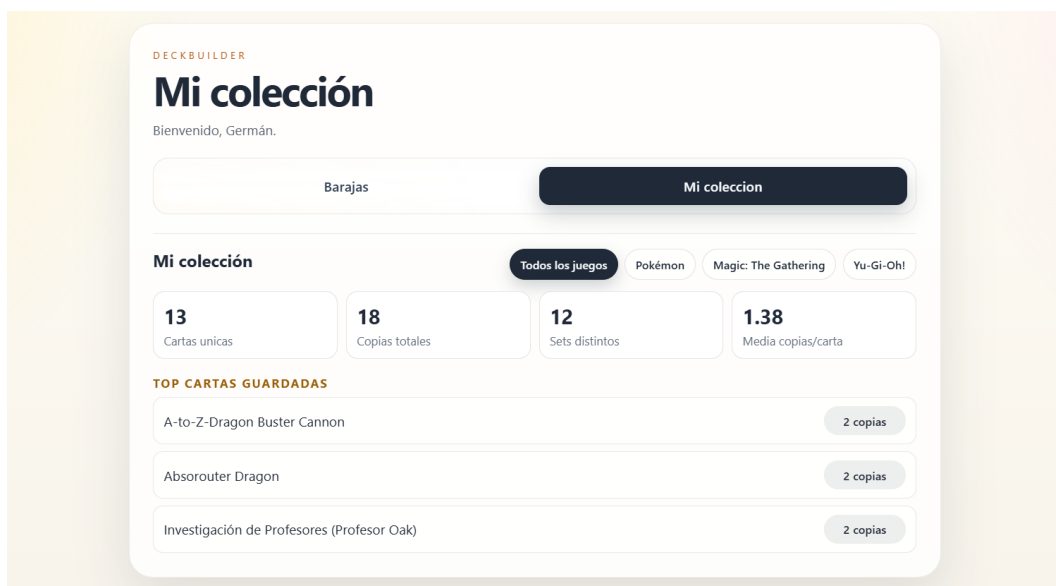
Tras la validación del acceso, se desarrollaron las vistas de colección y el detalle de cada baraja. Estas interfaces permiten al usuario interactuar en tiempo real con su contenido, editando la composición del mazo y persistiendo los cambios de forma inmediata con la API. Esta evolución incluye una refactorización integral del cliente por áreas de responsabilidad (autenticación, colección y comunicación con la API) para facilitar la escalabilidad futura.

Como fase de cierre de la interfaz, se incorporó la funcionalidad de borrado definitivo de barajas y se aplicaron técnicas de diseño responsive. Estas mejoras aseguran que la experiencia de usuario sea óptima y visualmente coherente en una amplia variedad de dispositivos.

Las principales pantallas del frontend pueden verse en la Figura 3.1, la Figura 3.2, la Figura 3.3 y la Figura 3.4.



**Figura 3.1:** Pantalla de colección de barajas



**Figura 3.2:** Pantalla de Mi colección

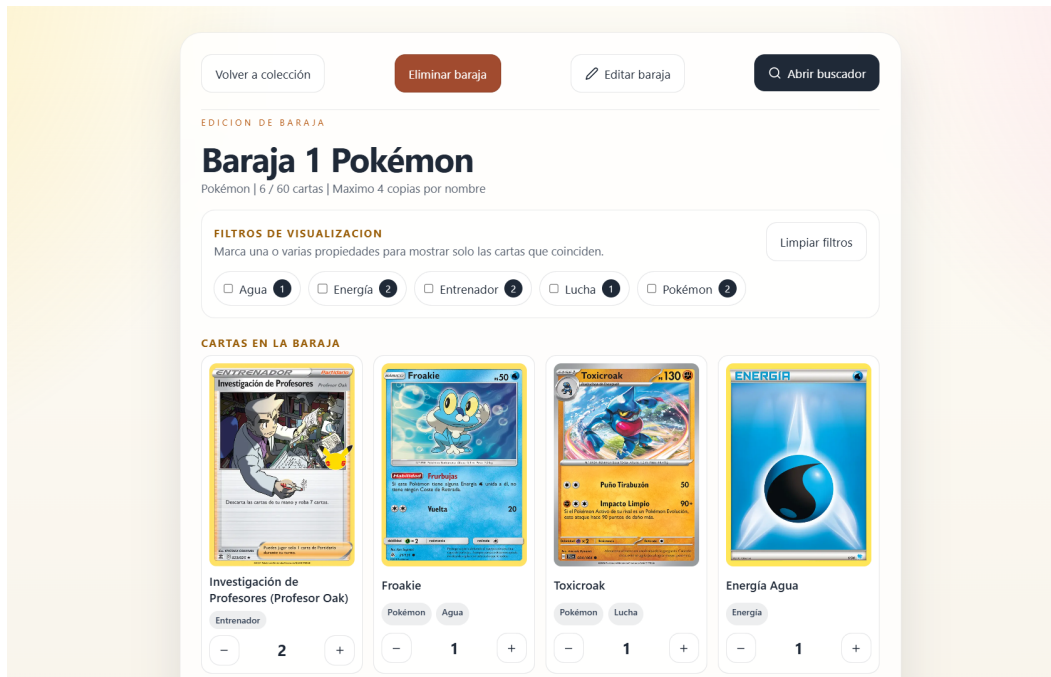


Figura 3.3: Pantalla de detalle y edición de baraja

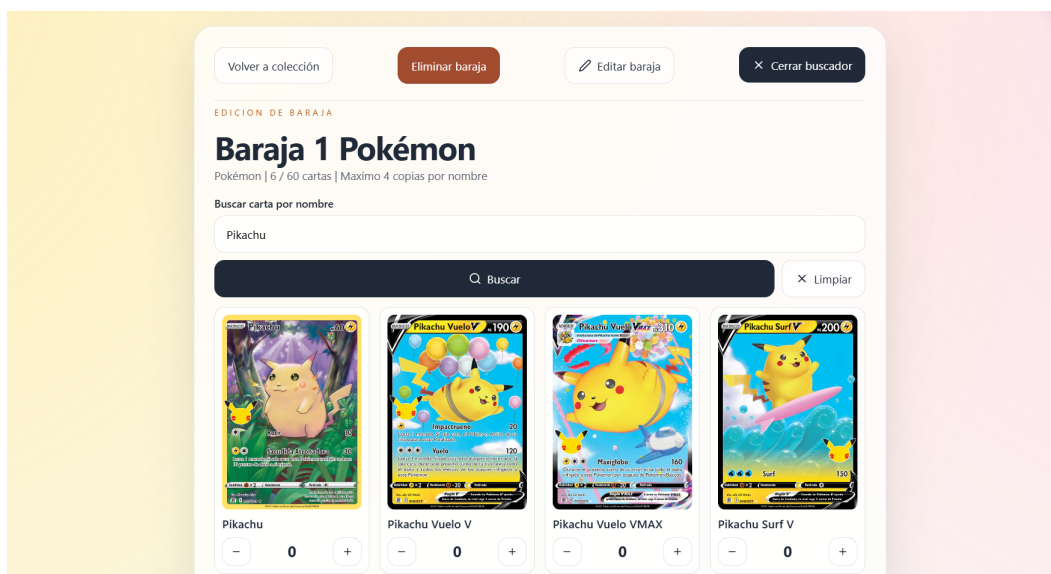


Figura 3.4: Pantalla de buscador de cartas

### 3.4. Capacidades multijuego y editor visual

Una de las partes más relevantes de esta fase fue la transformación de la plataforma en una solución multijuego auténtica. Se integra el tipo de juego (Pokémon, Magic o Yu-Gi-Oh!) como un atributo estructural dentro de la base de datos de MongoDB, permitiendo que el sistema se adapte dinámicamente según la elección del usuario al crear una nueva baraja.

Gracias a esta base polimórfica, se desarrolló un endpoint de búsqueda unificado capaz de consultar múltiples fuentes externas de manera homogénea. Esta integración permite al usuario buscar cartas de diferentes franquicias bajo un mismo estándar visual, simplificando la complejidad técnica de las diferentes estructuras de datos de origen.

Finalmente, se construyó el editor visual de barajas, la herramienta central de la aplicación. En este componente, los usuarios pueden añadir o retirar cartas mediante controles directos, mientras el sistema aplica automáticamente las reglas de validación específicas de cada juego dentro de la lógica de actualización, garantizando que el mazo resultante sea legal dentro de cada uno de los juegos.

El extracto 3.4 resume la base de la implementación multijuego: definir tipos de juego normalizados y seleccionar el proveedor de búsqueda en función del valor recibido.

---

```
1 public static class DeckGameTypes
2 {
3     public const string Pokemon = "pokemon";
4     public const string Magic = "magic";
5     public const string Yugioh = "yugioh";
6 }
7
8 var cards = normalizedGameType switch
9 {
10     DeckGameTypes.Pokemon => await SearchPokemonCardsAsync(httpClient,
11         query, cancellationToken),
12     DeckGameTypes.Magic => await SearchMagicCardsAsync(httpClient, query,
13         cancellationToken),
14     DeckGameTypes.Yugioh => await SearchYugiohCardsAsync(httpClient,
15         query, cancellationToken),
16     _ => []
17 };
```

---

**Extracto de código 3.4:** DeckGameTypes simplificado y selección de proveedor (C#)

## 3.5. Resultados de la fase

La Fase 2 concluyó exitosamente con un núcleo funcional operativo que integra autenticación, vinculación de datos por usuario, un CRUD completo y un motor multijuego. El sistema ha demostrado su funcionamiento al permitir la construcción de mazos completos de forma muy clara e interactiva.

Desde el punto de vista arquitectónico, la solución queda preparada para crecer sin necesidad de reestructurar la base de datos para cada nueva franquicia. Se ha logrado una separación clara entre la interfaz, la lógica de negocio y la capa de persistencia, dejando una base sólida para las próximas etapas.

## 4. Documentación y Presentación

---

### 4.1. Elaboración de la documentación técnica

La tercera fase del proyecto se ha dedicado íntegramente a la consolidación de toda la información generada durante el ciclo de vida del desarrollo para cumplir con los requisitos de evaluación de la asignatura. El núcleo de esta fase es la redacción de la presente memoria, un documento que no solo describe el producto final, sino que justifica cada decisión tomada por el equipo, desde la elección del stack tecnológico hasta la implementación de las reglas de negocio con las barajas.

Esta labor de documentación ha permitido realizar una reflexión crítica sobre el trabajo realizado, asegurando que todo lo alcanzado en las fases anteriores quede registrado. La memoria actúa como el soporte principal que demuestra la adquisición de competencias en el uso de C# y MongoDB, detallando cómo se ha resuelto el problema de la variabilidad de datos mediante un sistema CRUD eficiente y escalable.

### 4.2. Preparación de la presentación

Un componente fundamental de esta etapa ha sido la preparación de la presentación requerida para la asignatura. Este proceso ha consistido en sintetizar los aspectos más relevantes del desarrollo de DeckBuilder para su exposición. Se han diseñado materiales visuales que destacan los puntos fuertes de la aplicación, como la integración de las APIs externas y la flexibilidad de la base de datos documental.

El objetivo de esta tarea es comunicar de forma clara y concisa los resultados obtenidos y demostrar que el núcleo funcional desarrollado en la fase anterior es operativo y responde a los objetivos planteados al inicio.

### 4.3. Desarrollo del archivo README: Manual usuario y despliegue

Como cierre técnico del proyecto, se ha elaborado un archivo README dentro del repositorio del código fuente. Este documento se ha diseñado para cumplir una doble función: servir como manual de usuario y como guía detallada de despliegue e instalación. En él se explican paso a paso las instrucciones necesarias para que cualquier usuario pueda configurar el entorno de ejecución, incluyendo la

vinculación con MongoDB, la instalación de dependencias de .NET 8 y el arranque del frontend.

Además de las instrucciones técnicas, el README incluye una guía de uso que describe el flujo operativo de la aplicación y los enlaces de acceso al despliegue en Render.

## **4.4. Resultados finales de la fase**

La conclusión de esta fase marca el fin del desarrollo de DeckBuilder. Con la memoria técnica finalizada, la presentación preparada y el archivo README configurado, el proyecto queda listo para su entrega definitiva. El resultado es una aplicación que funciona de forma correcta, respaldada por una documentación para entender todo lo realizado.



## 5. Conclusiones

---

Tras la finalización del desarrollo de DeckBuilder, se ha logrado cumplir el objetivo principal de diseñar e implementar una aplicación funcional que utilice las tecnologías de C# y MongoDB.

Uno de los aspectos más relevantes del proyecto ha sido la utilización de una base de datos documental. MongoDB ha permitido gestionar de una manera eficiente la variabilidad entre los distintos juegos de cartas, evitando las limitaciones de los modelos relacionales tradicionales.

Desde el punto de vista técnico, el desarrollo de una API REST completa, junto con un frontend interactivo, ha permitido cubrir todo el ciclo de vida del usuario a lo largo de la aplicación, desde la autenticación hasta la gestión avanzada de barajas y colecciones. La integración con APIs externas aporta además un valor añadido claro a la aplicación.

En conclusión, el proyecto ha permitido aplicar de forma práctica los conceptos estudiados en la asignatura, especialmente en el uso de bases de datos NoSQL, consolidando así una base sólida tanto a nivel técnico como a nivel conceptual.

## 6. Declaración sobre el uso de herramientas de IA

---

Durante el desarrollo de este proyecto se han utilizado herramientas de inteligencia artificial como apoyo al trabajo realizado por los autores. En concreto, se ha recurrido a ChatGPT, Gemini y GitHub Copilot en distintas fases del proyecto, siempre como soporte y nunca como sustituto del criterio técnico o del trabajo de desarrollo, redacción y maquetación.

ChatGPT y Gemini se han empleado principalmente para:

- resolver dudas sobre el uso de tecnologías como MongoDB y C#;
- apoyar la redacción de partes de la memoria técnica, mejorando su claridad, estructura y redacción;
- proponer ideas para mejorar la lógica interna de la aplicación;
- generar ejemplos de peticiones y respuestas para entender mejor el funcionamiento de la API.

GitHub Copilot se ha utilizado durante el desarrollo del código y también en tareas de documentación. Su uso ha sido especialmente útil para:

- autocompletar funciones en C# y TypeScript;
- sugerir implementaciones en la lógica del backend y del frontend;
- acelerar la implementación del frontend, que no era un requisito inicial estricto del proyecto y se desarrolló como ampliación funcional;
- asistir en la redacción y revisión de la memoria técnica;
- facilitar el paso de una versión inicial redactada en Google Docs a su posterior maquetación en LaTeX.

Ejemplos de prompts utilizados:

- “Quiero que revises si la memoria cumple ahora mismo todos los requisitos que se piden en las instrucciones de entrega y en las cuestiones sobre los trabajos. Este es el README que hace de manual de despliegue y manual de usuario: README.md.”
- “Quiero utilizar componentes de react-icons para los botones de la app, por ejemplo el de cerrar sesión o el más y menos de las cartas.”
- “Quiero hacer un proyecto CRUD conectando C# (.NET) con MongoDB. Necesito ideas de proyecto que tengan sentido; el proyecto lo realizamos entre dos personas.”

Algunas conversaciones con ChatGPT:

- <https://chatgpt.com/share/69ee4684-5020-83eb-9fa2-355e1040c50c>
- <https://chatgpt.com/share/69ecf754-aeb0-8397-b190-0e320f49cae1>

En todo caso, el uso de estas herramientas se ha limitado a tareas de apoyo, revisión y sugerencia. Las decisiones finales de diseño, implementación, redacción y maquetación han sido tomadas por los autores del proyecto.

# Bibliografía

---